Manual RPI Status Dashboard

Instalación, seguridad y operación

Proyecto en desarrollo · versión documental del 7 de noviembre de 2026

Aviso: la integración, la construcción de la imagen y el despliegue en la Raspberry Pi siguen pendientes de verificación. Este manual describe el código inspeccionado, no una instalación de producción validada. Lea las limitaciones antes de ejecutar comandos.

Índice

1 Alcance y estado del proyecto 2
2 Arquitectura y flujo de datos 2
3 Implementación y limitaciones actuales 2
4 Requisitos y comprobaciones previas 2
5 Configuración del entorno 3
6 Instalación prevista 3
7 Operación con scripts existentes 3
8 API y consultas de diagnóstico 3
9 Historial y límites de datos 4
10 Copias de seguridad y recuperación 4
11 Seguridad del host y de la aplicación 4
12 Cloudflare Tunnel y Access 4
13 Diagnóstico de incidencias 4
14 Desarrollo y pruebas pendientes 5
15 Edición y generación de este manual 5

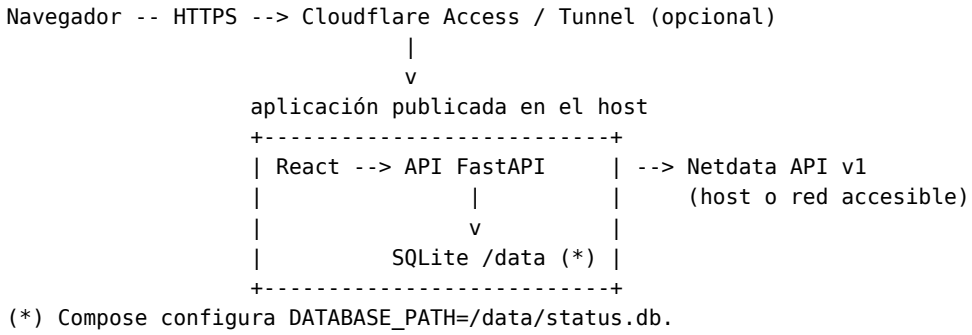
1 Alcance y estado del proyecto

El objetivo es visualizar información de Netdata mediante una interfaz React y un backend FastAPI desplegados con Docker Compose. El frontend se está alineando con el backend; su integración completa sigue pendiente de validación. No se ha comprobado una instalación integral en la Raspberry Pi ni el acceso a Netdata real desde el contenedor.

El diagnóstico `docs/DIAGNOSTICO.md` corresponde al equipo de desarrollo x86_64, no al host objetivo. Allí no se podía acceder al daemon Docker. Consulte también `LIMITACIONES-INFRA.md`, `backend/README.md` y los archivos efectivos, que pueden cambiar durante el desarrollo.

2 Arquitectura y flujo de datos

El siguiente diagrama es conceptual; las flechas representan conexiones previstas, no integración verificada.



Compose define un servicio `app` y un volumen `app_data` montado en `/data`. No configura Cloudflare ni Netdata. La publicación predeterminada es `127.0.0.1:8765` del host hacia el puerto 8765 del contenedor. Las métricas del contenedor no equivalen necesariamente a las del anfitrión: los datos no expuestos o no verificables son **no disponibles**, no cero.

3 Implementación y limitaciones actuales

El Dockerfile inicia `uvicorn app:app` desde `/app/backend`. Compila el frontend y copia los estáticos a `/app/frontend/dist` FastAPI los sirve cuando ese directorio existe. Compose configura `DATABASE_PATH=/data/status.db` en el volumen `app_data`, ruta utilizada por el backend y los scripts de respaldo y restauración.

El frontend consulta el catálogo y datos de Netdata a través de la API y sigue en desarrollo. Una prueba local con fuente simulada verificó compilación, gráfica y navegación en escritorio y móvil; su compatibilidad con Netdata real sigue pendiente. El healthcheck comprueba solo el proceso API, no Netdata. Compose usa por defecto `http://host.docker.internal:19999` si Netdata escucha solo en loopback del host no será accesible por esa dirección. No se modifican automáticamente Netdata ni el firewall.

La navegación ofrece Resumen, Rendimiento, Temperatura, Memoria, Almacenamiento, Red, Docker, Servicios, Históricos, Alertas y Configuración. Las secciones filtran gráficos disponibles; Históricos muestra instantáneas guardadas y Alertas incluye transiciones de reglas. El inventario de contenedores y servicios y la edición visual de configuración aún no están implementados. Un gráfico vacío no demuestra que el recurso no exista.

4 Requisitos y comprobaciones previas

En la Raspberry Pi confirme arquitectura, espacio, Docker, Compose, acceso autorizado al daemon, estado de Netdata y disponibilidad de los puertos previstos. El acceso al daemon de Docker es privilegiado. No monte su socket en la aplicación ni utilice contenedores privilegiados.

```
uname -m
df -h
docker version
docker compose version
ss -lnt
```

Estas consultas pueden fallar por permisos; no demuestran que un servicio remoto esté disponible. No ejecute comandos de instalación hasta revisar la configuración y los requisitos. El servidor objetivo y su red requieren autorización del responsable.

5 Configuración del entorno

`.env.example` define `PORT=8765`, `NETDATA_URL=http://host.docker.internal:19999`, `NETDATA_CHARTS`, `POLL_INTERVAL_SECONDS=30`, `HISTORY_RETENTION_DAYS=30` y `TZ`. Compose publica exclusivamente en loopback y fija `DATABASE_PATH=/data/status.db`. El instalador crea `.env` si falta y **continúa**; prepare y revise el archivo antes de lanzarlo si necesita otros valores.

El backend admite `NETDATA_URL`, `NETDATA_TOKEN`, `DATABASE_PATH`, `NETDATA_CHARTS`, `POLL_INTERVAL_SECONDS`, `HISTORY_RETENTION_DAYS` y `ALERT_RULES`. Fuera de Compose, sus valores predeterminados pueden diferir; consulte `backend/app.py`. No incluya credenciales en URL, repositorio ni registros. La autenticación Bearer opcional debe corresponder a la configuración real de Netdata.

6 Instalación prevista

Con Docker accesible y autorización, prepare primero la configuración desde la raíz del proyecto (no sobrescriba un `.env` existente):

```
# Solo si .env no existe:
(umask 077; cp -n .env.example .env)
# Revise .env antes de continuar.
./install.sh
```

`install.sh` delega en `scripts/install.sh`: requiere Linux, Python 3, Docker y daemon accesible; valida Compose y puerto loopback libre, ejecuta `up -d --build` y llama a `verify.sh`. Si falta `.env`, lo crea y continúa; no sobrescribe uno existente. Puede construir imágenes y crear o actualizar recursos del proyecto. El preflight del puerto no es válido para un daemon remoto y existe una carrera antes de publicar. No instala Docker ni configura túneles o firewall. Construcción y arranque se verificaron en el PC amd64; ARM64 sigue pendiente.

También se puede iniciar con `./deploy.sh`. Para ensayar la infraestructura sin tocar los datos del despliegue, ejecute `./scripts/test-deployment.sh`. Este crea un proyecto Compose con puerto y volumen temporales, verifica salud, API, frontend y persistencia tras reinicio, y elimina sus recursos de prueba. Se ejecutó correctamente en amd64; no comprueba Netdata real, Cloudflare, ARM64 ni la restauración.

7 Operación con scripts existentes

Los scripts siguientes requieren `.env`, Docker y el complemento Compose. Se ejecutan desde la raíz y operan sobre el proyecto Compose configurado:

```
./scripts/start.sh    # inicia o crea servicios
./scripts/status.sh   # estado Compose
./scripts/logs.sh     # últimos 100 registros de app y seguimiento
./scripts/restart.sh  # reinicia servicios
./scripts/verify.sh   # valida Compose y consulta API y / desde app
./scripts/stop.sh     # detiene servicios
```

Interrumpa el seguimiento de registros con Ctrl+C; no divulgue salidas sensibles. `stop.sh` detiene los servicios sin eliminar contenedores, red ni volúmenes. `start.sh` puede crear o iniciar recursos; no realiza una construcción explícita. `verify.sh` no comprueba Netdata ni confirma métricas reales. Los scripts no constituyen por sí solos una prueba integral.

8 API y consultas de diagnóstico

El backend implementa `GET /api/health`, `/api/charts`, `/api/data`, `/api/alerts`, `/api/history`, `/api/rule-alerts` y `/api/rule-alerts/history`. `/api/health` solo refleja el proceso, no comprueba Netdata. Por ejemplo, **si** el backend ya está accesible localmente en el puerto configurado:

```
curl --fail http://127.0.0.1:8765/api/health
curl --fail http://127.0.0.1:8765/api/charts
curl --fail 'http://127.0.0.1:8765/api/history?kind=alerts&limit=10'
curl --fail http://127.0.0.1:8765/api/rule-alerts
```

Los identificadores de gráficos deben obtenerse de la instalación real. Para `/api/data` se requiere `chart` las respuestas corresponden a Netdata API v1. La documentación interactiva del backend está en `/docs` cuando se ejecuta. La API carece de autenticación propia: protéjala antes de cualquier exposición.

9 Historial y límites de datos

El backend consulta periódicamente las alertas y los gráficos de `NETDATA_CHARTS` y de las reglas configuradas en `ALERT_RULES`. Si no hay gráficos ni reglas configurados, no recoge gráficos. Guarda una instantánea de un punto por gráfico y las respuestas válidas de alertas. La retención se aplica durante la recogida. No es una serie completa ni envía notificaciones. Fallos o reinicios pueden dejar huecos y múltiples instancias generar duplicados.

`ALERT_RULES` es un array JSON, vacío por defecto; cada regla requiere `id`, `chart`, `dimension`, `operator` (`gt` o `lt`) y `threshold` `duration_seconds` es opcional. El backend registra cambios de estado de reglas locales y ofrece sus estados e historial en `/api/rule-alerts` y `/api/rule-alerts/history`. Un dato ausente o antiguo produce estado desconocido; estas reglas no sustituyen las alarmas originales de Netdata. Verifique IDs y unidades reales.

Compruebe unidades, cobertura y procedencia de cada dato con Netdata real; no interprete valores vacíos como métricas del host. La persistencia en `/data` y su recuperación deben verificarse en el entorno objetivo.

10 Copias de seguridad y recuperación

```
./scripts/backup.sh
```

El script requiere contenedor `app` en ejecución y una base existente en la ruta absoluta `DATABASE_PATH`, fijada por Compose en `/data/status.db`. Usa SQLite Online Backup y guarda un archivo privado con fecha en `backups/` no copia secretos ni otros archivos.

La restauración sustituye los datos actuales y requiere autorización explícita:

```
./scripts/restore.sh --confirm backups/archivo.sqlite3
```

El script valida integridad y tabla `snapshots`, crea un backup previo con la aplicación activa, detiene `app`, restaura mediante SQLite Backup y vuelve a iniciar los servicios. Si falla después de detenerlos, pueden quedar detenidos. Proteja las copias y pruebe la recuperación aisladamente; no se ha verificado con Docker ni en producción.

11 Seguridad del host y de la aplicación

Mantenga el sistema y dependencias actualizados según la política del entorno. Restrinja las interfaces de escucha y el acceso al backend y a Netdata; revise reglas efectivas, no solo archivos de configuración. La aplicación no proporciona autenticación propia ni TLS. El Compose inspeccionado limita capacidades y usa `no-new-privileges`, pero esto no sustituye controles de red ni autorización.

Guarde secretos con permisos mínimos, no los imprima en diagnósticos y rote cualquier credencial expuesta. Docker no debe montar el socket en el contenedor de aplicación. No modifique servicios, redes, firewall, túneles ni volúmenes ajenos al proyecto.

12 Cloudflare Tunnel y Access

Antes de publicar la ruta del túnel, cree la aplicación Access para el hostname con políticas explícitas de identidad y mínimo privilegio: sin Access la aplicación publicada podría quedar accesible en Internet. Active **Protect with Access** en cloudflared para validar el token JWT de Access, o implemente la validación de tokens en el origen. No confíe ciegamente en cabeceras del cliente.

Pruebe que una identidad no autorizada sea rechazada y que una autorizada pueda entrar; compruebe también el rechazo de solicitudes sin token válido donde corresponda. Un túnel no habilita Access automáticamente. Protect with Access en cloudflared no protege rutas directas que lo eviten: restrinja también el acceso directo a la IP, la LAN y otros hostnames. Estas medidas son recomendaciones, no una configuración verificada. Referencia: documentación oficial de Cloudflare.

Si cloudflared usa `localhost:8765` para conectar con el origen en el host, debe compartir su namespace de red. En otro contenedor bridge, `localhost` apunta a ese contenedor y un puerto publicado solo en loopback del host no es accesible automáticamente. Evalúe la topología autorizada; no cambie la escucha a `0.0.0.0` sin evaluación de seguridad y autorización. No se ha verificado ni modificado la configuración Cloudflare de la Raspberry.

13 Diagnóstico de incidencias

- **El contenedor no arranca:** revise `./scripts/status.sh` y `./scripts/logs.sh`, la configuración y los permisos de SQLite.

- **La interfaz falla:** compruebe que se haya compilado y servido el directorio estático, las peticiones del navegador y la compatibilidad de respuestas; el frontend sigue en desarrollo.
- **La API responde 502:** compruebe Netdata, URL, credenciales y conectividad desde el namespace del backend. Evite exponer el servicio para solucionar el problema.
- **No hay historial:** revise IDs, intervalo, ruta de base, permisos y disponibilidad de Netdata. La salud del proceso no garantiza datos.
- **Falla el respaldo:** verifique que `app` esté en ejecución y que `DATABASE_PATH` identifique la base existente que usa el backend.

14 Desarrollo y pruebas pendientes

El backend documenta pruebas unitarias en `backend/README.md` consulte ese archivo para sus resultados y límites. Para ejecutarlas en un entorno Python apropiado, desde la raíz:

```
python -m unittest discover -s backend/tests -v
```

Quedan pendientes las pruebas integradas con Netdata real, la restauración verificada en Docker, la operación ARM64 y las comprobaciones de seguridad en la Raspberry objetivo. La persistencia tras reinicio se verificó en un proyecto de prueba amd64 aislado. No existe un script de despliegue Cloudflare; no se debe asumir su disponibilidad.

La actualización y retirada requieren evaluar su impacto y autorización:

```
./scripts/update.sh             # backup, build, up y verify; no descarga código
./scripts/uninstall.sh --confirm # compose down; conserva volúmenes y backups
```

Si falla el backup, la actualización se detiene; no hay rollback automático si fallan pasos posteriores. La retirada elimina contenedores y red del proyecto, pero conserva volúmenes y copias. Se ha probado el backup en Docker amd64; actualización, retirada y restauración siguen pendientes de prueba.

15 Edición y generación de este manual

Edite `docs/manual.typ` y ejecute desde la raíz:

```
./scripts/generar_manual.sh
# Alternativa con ejecutable local explícito:
TYPST_BIN=/ruta/a/typst ./scripts/generar_manual.sh
pdfinfo docs/MANUAL_RPI_STATUS_DASHBOARD.pdf
pdftotext docs/MANUAL_RPI_STATUS_DASHBOARD.pdf -
```

El script usa primero `TYPST_BIN` si se define, después Typst en `PATH` si ninguno está disponible, intenta usar Docker con una imagen Typst ya presente localmente. La imagen predeterminada es `ghcr.io/typst/typst:latest`, modificable mediante `TYPST_DOCKER_IMAGE`. El contenedor no tiene red, no descarga imágenes y solo monta el archivo fuente en lectura y un directorio temporal de salida. Requiere daemon accesible y una imagen compatible. La salida `docs/MANUAL_RPI_STATUS_DASHBOARD.pdf` se reemplaza; el script no instala dependencias.